

BỘ CÔNG THƯƠNG
TRƯỜNG ĐẠI HỌC CÔNG NGHIỆP THÀNH PHỐ HỒ CHÍ MINH
KHOA CÔNG NGHỆ ĐIỆN TỬ



BÁO CÁO NGHIÊN CỨU
TUẦN 03
KHÓA LUẬN TỐT NGHIỆP

SINH VIÊN THỰC HIỆN: **Nguyễn Minh Phương**
Nguyễn Hữu Hưng Thịnh

Thành phố Hồ Chí Minh, tháng 04 năm 2026

Kiểm tra robot di động tự hành với ROS 2, Gazebo, SLAM Toolbox và Nav2 trên nền tảng mô phỏng

MỞ ĐẦU

Báo cáo này trình bày quá trình xây dựng hệ thống mô phỏng robot di động tự hành trên nền tảng ROS 2 Jazzy, Gazebo và RViz2. Hệ thống sử dụng SLAM Toolbox để tạo bản đồ từ dữ liệu LiDAR mô phỏng và sử dụng Nav2 để định vị, lập kế hoạch đường đi và điều hướng robot đến vị trí mong muốn.

Phạm vi báo cáo tuần này tập trung vào mô phỏng phần mềm, bao gồm xây dựng mô hình robot, môi trường Gazebo, cấu hình cảm biến, bridge dữ liệu Gazebo - ROS 2, tạo bản đồ bằng SLAM và điều hướng bằng Nav2. **Hệ thống chưa triển khai trên robot thực tế, chưa tích hợp LiDAR thật, chưa kiểm chứng với sai số phần cứng và chưa tối ưu sâu thuật toán planning trong môi trường phức tạp.**

Mô phỏng được luồng hoạt động cơ bản của robot tự hành, giúp kiểm thử thuật toán và cấu hình trước khi triển khai phần cứng. Nhược điểm là kết quả mô phỏng chưa phản ánh đầy đủ các yếu tố thực tế như nhiễu cảm biến, trượt bánh, sai số odometry và giới hạn phần cứng.

Hệ thống này đóng vai trò là nền tảng ban đầu để phát triển robot tự hành thực tế trong tương lai, có thể mở rộng sang tích hợp LiDAR thật, encoder, điều khiển động cơ, robot phần cứng và các thuật toán điều hướng nâng cao.

Link demo kết quả: <https://youtu.be/6uUdpF2ltL4>

I. TỔNG QUAN HỆ THỐNG MÔ PHỎNG

Hệ thống được xây dựng trên ROS 2 Jazzy kết hợp Gazebo và RViz2. Gazebo dùng để mô phỏng môi trường, robot, cảm biến LiDAR và chuyển động. RViz2 dùng để quan sát dữ liệu ROS 2 như bản đồ, scan LiDAR, TF, vị trí robot và đường đi điều hướng.

File **backup.launch.py** có vai trò khởi động mô phỏng chính: đọc robot từ file **robot.urdf.xacro**, chạy **robot_state_publisher**, mở Gazebo với world **small_map_with_vehicle_blue.sdf**, spawn robot vào Gazebo, mở RViz2 và bridge các topic quan trọng như **/scan**, **/cmd_vel**, **/odometry**, **/tf**, **/joint_states** từ Gazebo sang ROS 2.

Thành phần	Vai trò trong hệ thống
Gazebo	Mô phỏng môi trường, robot, cảm biến và chuyển động.
RViz2	Trực quan hóa dữ liệu ROS 2 như bản đồ, LaserScan, TF, robot model và đường đi.
SLAM Toolbox	Tạo bản đồ môi trường từ dữ liệu LiDAR mô phỏng.

Nav2	Định vị, lập kế hoạch đường đi và điều hướng robot đến goal.
ros_gz_bridge	Chuyển đổi dữ liệu giữa Gazebo và ROS 2 thông qua các topic.

II. CƠ SỞ MÔ TẢ ROBOT TRONG ROS 2

2.1. URDF

URDF (Unified Robot Description Format) là định dạng XML dùng để mô tả cấu trúc hình học, liên kết, khớp và thuộc tính vật lý của robot. Trong hệ thống ROS 2, URDF giúp các node biết robot gồm những link nào, các link được nối với nhau bằng joint nào và quan hệ tọa độ giữa các bộ phận ra sao.

- link: mô tả một bộ phận của robot, gồm hình dạng, quán tính và va chạm.
- joint: mô tả khớp nối giữa các link.
- transmission: mô tả truyền động nếu hệ thống sử dụng ros2_control.
- sensor và plugin: dùng khi mô phỏng robot trong Gazebo.

Khi load URDF vào RViz2, RViz2 có thể hiển thị mô hình 3D của robot và quan sát cây TF giữa các link. Dữ liệu TF này rất quan trọng cho SLAM và Navigation.

2.2. Xacro

Xacro (XML Macros) là cách viết URDF có hỗ trợ biến, macro và include file. Thay vì viết toàn bộ robot trong một file URDF dài, Xacro cho phép tách robot thành nhiều file nhỏ, dễ quản lý và dễ tái sử dụng hơn.

2.3. Sensor

Trong mô hình robot, mỗi sensor thường được định nghĩa như một link riêng vì cảm biến có vị trí và hệ tọa độ riêng. Sensor được gắn vào base_link hoặc chassis_link thông qua joint kiểu fixed. URDF/Xacro chủ yếu mô tả hình dáng, vị trí và TF của sensor; để sensor phát dữ liệu như LaserScan cần sử dụng plugin Gazebo hoặc node ROS mô phỏng cảm biến.

2.4. Gazebo control

File **gazebo_control.xacro** dùng để cấu hình phần điều khiển robot trong môi trường mô phỏng Gazebo. Nếu **robot.urdf.xacro** mô tả cấu trúc cơ khí và **sensor.xacro** mô tả cảm biến, thì **gazebo_control.xacro** có nhiệm vụ kết nối mô hình robot với hệ thống điều khiển trong Gazebo/ROS 2.

Trong mô phỏng, robot không thể tự di chuyển nếu chỉ có hình dạng và khớp nối. Để robot nhận lệnh vận tốc và xuất dữ liệu odometry, cần khai báo plugin điều khiển cho Gazebo. Plugin này thường nhận lệnh từ topic **/cmd_vel**, mô phỏng chuyển động robot và xuất dữ liệu **/odometry** cùng TF.

Thành phần	Ý nghĩa
------------	---------

/cmd_vel	Topic nhận lệnh vận tốc từ teleop hoặc Nav2.
Gazebo control plugin	Chuyển lệnh vận tốc thành chuyển động của robot trong Gazebo.
/odometry	Xuất vị trí, hướng và vận tốc tương đối của robot.
/tf	Cung cấp quan hệ tọa độ giữa các frame như odom, base_link, lidar_link.

III. XÂY DỰNG MÔ HÌNH ROBOT VÀ MÔI TRƯỜNG MÔ PHỎNG

Cấu trúc workspace:

dev_ws/

├── src/

└── my_robot_description/

├── launch/

| ├── backup.launch.py

| ├── slam.launch.py

| └── nav.launch.py

├── urdf/

| ├── robot.urdf.xacro

| ├── sensor.xacro

| └── gazebo_control.xacro

├── worlds/

| └── small_map_with_vehicle_blue.sdf

├── maps/

| ├── small_map.pgm

| ├── small_map.yaml

| └── createMAP.txt

├── config/

└── slam_toolbox.yaml

└─ nav2_params.yaml

3.1. Định nghĩa mô hình robot

Mô hình robot được xây dựng bằng các file Xacro để tách phần thân robot, cảm biến và điều khiển mô phỏng. Cách tách file này giúp báo cáo và mã nguồn dễ quản lý hơn.

File	Vai trò
robot.urdf.xacro	File chính định nghĩa cấu trúc robot, gồm thân xe, bánh xe, joint, kích thước, khối lượng và liên kết giữa các bộ phận.
sensor.xacro	Định nghĩa cảm biến của robot, đặc biệt là LiDAR dùng để quét môi trường và xuất dữ liệu /scan.
gazebo_control.xacro	Định nghĩa plugin điều khiển trong Gazebo, nhận lệnh vận tốc từ /cmd_vel và xuất odometry cho robot.

3.2. Xây dựng môi trường Gazebo

Môi trường mô phỏng được định nghĩa bằng file world dạng SDF. Trong báo cáo này, world sử dụng là **small_map_with_vehicle_blue.sdf**. File SDF này định nghĩa không gian mô phỏng, vật cản, bản đồ và khu vực robot di chuyển. Khi chạy launch, Gazebo mở world này để robot hoạt động trong môi trường giả lập.

3.3. Xây dựng file launch mô phỏng

File **backup.launch.py** có nhiệm vụ khởi động toàn bộ phần mô phỏng cơ bản. File này xử lý **robot.urdf.xacro** thành URDF, chạy **robot_state_publisher** để publish trạng thái robot, mở Gazebo với world đã cấu hình, spawn robot vào môi trường, mở RViz2 và tạo bridge giữa Gazebo với ROS 2.

Các topic chính được bridge gồm /scan, /cmd_vel, /odometry, /tf và /joint_states. Nhờ đó, dữ liệu từ Gazebo có thể được sử dụng bởi các node ROS 2 như SLAM Toolbox và Nav2.

IV. QUY TRÌNH TẠO BẢN ĐỒ BẰNG SLAM

4.1. Cấu hình SLAM Toolbox

File **slam_toolbox.yaml** chứa các tham số cấu hình cho SLAM Toolbox, bao gồm frame của bản đồ, frame odometry, frame robot, topic LiDAR, độ phân giải bản đồ và giới hạn khoảng đo của LiDAR.

Tham số	Ý nghĩa
map_frame: map	Frame bản đồ toàn cục.
odom_frame: odom	Frame odometry.
base_frame: base_link	Frame gốc của robot.
scan_topic: /scan	Topic dữ liệu LiDAR.
mode: mapping	Chế độ tạo bản đồ.
resolution: 0.05	Độ phân giải bản đồ.
min_laser_range: 0.05	Khoảng đo nhỏ nhất của LiDAR.

max_laser_range: 12.0	Khoảng đo lớn nhất của LiDAR.
-----------------------	-------------------------------

4.2. File launch SLAM

File **slam.launch.py** có nhiệm vụ khởi động SLAM Toolbox ở chế độ online async. Node này nhận dữ liệu **/scan** từ LiDAR, kết hợp với **/odometry** và TF để ước lượng vị trí robot trong khi di chuyển, đồng thời xây dựng bản đồ môi trường và publish bản đồ lên topic **/map**.

4.3. Quy trình chạy tạo map

- Chạy mô phỏng robot và môi trường bằng backup.launch.py.
- Chạy SLAM Toolbox bằng slam.launch.py.
- Điều khiển robot bằng teleop để LiDAR quét toàn bộ môi trường.
- Lưu bản đồ bằng map_saver_cli sau khi bản đồ đã tương đối đầy đủ.

```
ros2 launch my_robot_description backup.launch.py
ros2 launch my_robot_description slam.launch.py
ros2 run teleop_twist_keyboard teleop_twist_keyboard
ros2 run nav2_map_server map_saver_cli -f small_map
```

Kết quả thu được gồm hai file bản đồ: **small_map.pgm** và **small_map.yaml**. Hai file này được lưu trong thư mục maps để sử dụng cho bước điều hướng bằng Nav2.

V. ĐIỀU HƯỚNG TỰ ĐỘNG BẰNG NAV2

5.1. Cấu hình Nav2

File **nav2_params.yaml** chứa cấu hình cho hệ thống điều hướng Nav2, bao gồm map_server, AMCL, planner_server, controller_server, costmap, behavior_server và velocity_smoother.

Thành phần	Vai trò
map_server	Đọc bản đồ đã lưu từ small_map.yaml.
amcl	Định vị robot trên bản đồ.
planner_server	Tạo đường đi toàn cục.
controller_server	Sinh lệnh vận tốc để robot bám theo đường đi.
local_costmap	Bản đồ vật cản cục bộ quanh robot.
global_costmap	Bản đồ toàn cục dùng để lập kế hoạch.
bt_navigator	Quản lý hành vi điều hướng đến goal.
velocity_smoother	Làm mượt lệnh vận tốc trước khi gửi đến robot.

5.2. File launch Nav2

File **nav.launch.py** khởi động các node chính của Nav2 như map_server, amcl, planner_server, controller_server, bt_navigator, behavior_server, smoother_server, waypoint_follower và velocity_smoother. Ngoài ra, file này còn khởi động lifecycle manager để tự động chuyển các node Nav2 sang trạng thái active.

5.3. Quy trình chạy điều hướng

```
ros2 launch my_robot_description backup.launch.py
ros2 launch my_robot_description nav.launch.py
```

Trong RViz2, đặt **Fixed Frame** = **map** và thêm các thành phần quan sát như Map, TF, RobotModel, LaserScan, Odometry, Path và PoseWithCovariance. Sau đó dùng **2D Pose Estimate** để đặt vị trí ban đầu của robot, tiếp theo dùng **Nav2 Goal** để chọn điểm đích. Nav2 sẽ lập kế hoạch đường đi và gửi lệnh vận tốc qua **/cmd_vel** để robot di chuyển trong Gazebo.

VI. LUỒNG DỮ LIỆU HỆ THỐNG

6.1. Luồng dữ liệu khi tạo bản đồ

```
Gazebo
  ↓
Robot + LiDAR
  ↓
/scan + /odometry + /tf
  ↓
SLAM Toolbox
  ↓
/map
  ↓
map_saver_cli
  ↓
maps/small_map.pgm + maps/small_map.yaml
```

6.2. Luồng dữ liệu khi điều hướng

```
maps/small_map.yaml
  ↓
map_server
  ↓
/map
  ↓
AMCL + /scan + /odometry + /tf
  ↓
planner_server
  ↓
controller_server
  ↓
velocity_smoother
  ↓
/cmd_vel
  ↓
Gazebo robot
```

VII. KẾT QUẢ, HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN

7.1. Kết quả trong tuần

- Xây dựng được mô hình robot bằng URDF/Xacro và mô phỏng trong Gazebo.
- Cấu hình được cảm biến LiDAR mô phỏng và bridge dữ liệu /scan sang ROS 2.
- Chạy được quy trình tạo bản đồ bằng SLAM Toolbox.
- Lưu được bản đồ dưới dạng small_map.pgm và small_map.yaml.
- Cấu hình được Nav2 để sử dụng bản đồ đã lưu cho điều hướng cơ bản.
- Quan sát được dữ liệu mô phỏng và điều hướng trong RViz2.

7.2. Hạn chế hiện tại

- Hệ thống hiện mới dừng ở mức mô phỏng, chưa triển khai trên robot thật.
- Chưa tích hợp LiDAR thật, encoder thật và bộ điều khiển động cơ thực tế.
- Chưa kiểm chứng sai số do nhiều cảm biến, trượt bánh hoặc độ trễ phần cứng.
- Khả năng planning và điều hướng mới ở mức cơ bản, chưa tối ưu cho môi trường phức tạp hoặc có vật cản động.
- Chưa đánh giá định lượng độ chính xác của bản đồ và hiệu quả điều hướng.

7.3. Hướng phát triển tiếp theo

- Tích hợp LiDAR thật và kiểm tra dữ liệu /scan trên robot thực tế.
- Kết nối encoder, driver động cơ và bộ điều khiển vận tốc cho robot thật.
- Hiệu chỉnh lại TF, odometry và footprint để tăng độ ổn định khi điều hướng.
- Nâng cấp cấu hình Nav2, costmap và planner để robot di chuyển ổn định hơn.
- Bổ sung đánh giá thực nghiệm về thời gian đi đến goal, sai số vị trí và khả năng tránh vật cản.

TÀI LIỆU THAM KHẢO

- [1] ROS 2 Documentation, “URDF Tutorials”, Open Robotics.
- [2] ROS 2 Documentation, “Using a URDF in Gazebo”, Open Robotics.
- [3] ROS 2 Documentation, “Using URDF with robot_state_publisher”, Open Robotics.
- [4] ROS 2 Control Documentation, “ros2controlcli and controller_manager”.
- [5] ROS 2 Controllers Documentation, “diff_drive_controller”.
- [6] ROS 2 Package Documentation, “slam_toolbox”.
- [7] Navigation2 Documentation, “Nav2 Concepts and Tutorials”.